DATABRICKS BOOTCAMP · DAY 1 HOMEWORK

Lakebase Support Ticketing System

A full-stack internal support ticketing application built on **Databricks Lakebase** (managed Postgres OLTP) and deployed as a **Databricks App** — covering schema design, an OAuth-secured data layer, a corporate-grade Gradio UI, and end-to-end smoke testing.

Author	Jayanth Dolai (Senior Software Engineer, UnitedHealth Group / Optum)
Live App URL	https://support-ticketing-7474654640109575.aws.databricksapps.com
GitHub Repo	github.com/demonjd2026-afk/lakebase-support-ticketing
Platform	Databricks Free Edition (AWS) · Lakebase Postgres · Databricks Apps
Frontend	Gradio (Python) — 3-tab corporate ticketing portal
Auth Model	Service-principal OAuth (client_credentials) → Lakebase JWT

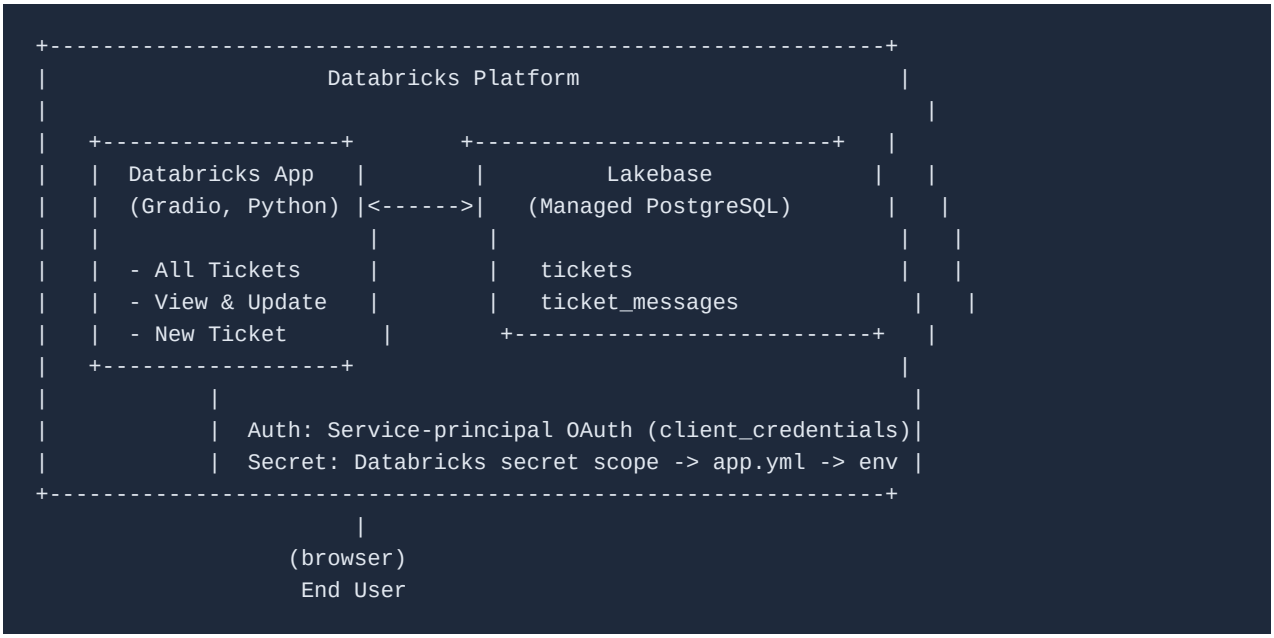
Project Highlights

- Two related Postgres tables (**tickets**, **ticket_messages**) with a foreign key and cascading delete
- 4+ seeded tickets across 3 statuses, 11+ threaded messages
- Full CRUD: view, filter, create, update status, add message, delete-with-confirmation
- Bonus features: priority/category fields, live dashboard stats, input validation, styled UI
- Secrets never hit source control — Lakebase credentials come from a Databricks secret scope, injected via **app.yml** and exchanged for a fresh OAuth token on every request

Contents

1 · Architecture Overview

The application follows a simple three-tier design: a Gradio frontend running as a Databricks App, a thin Python data-access layer, and Lakebase — Databricks' managed PostgreSQL OLTP engine — as the system of record.



Why Lakebase instead of a Delta table?

Dimension	Delta Lake (Analytics)	Lakebase (OLTP)
Designed for	Batch reads, BI, ML	Transactional reads/writes
Latency	Seconds to minutes	Milliseconds
Consistency	Eventually consistent	ACID, row-level
Write pattern	Append / bulk merge	Insert / Update / Delete
Best fit	Dashboards, pipelines	Apps, APIs, operational data

PHASE 1

Lakebase Schema & Seed Data

Provisioned a Lakebase project, defined two related Postgres tables, and seeded realistic sample data — all executed directly in the Lakebase SQL Editor.

Lakebase Project Configuration

Project	support-ticketing
Branch	production
Database	databricks_postgres
Auth type	OAuth (Databricks identities & service principals)

Schema DDL

```
CREATE TABLE tickets (
    ticket_id   SERIAL       PRIMARY KEY,
    title       TEXT         NOT NULL,
    status      TEXT         NOT NULL DEFAULT 'open',
    priority    TEXT         NOT NULL DEFAULT 'medium',
    category    TEXT,
    created_by  TEXT         NOT NULL,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE TABLE ticket_messages (
    message_id   SERIAL       PRIMARY KEY,
    ticket_id    INT          NOT NULL REFERENCES tickets(ticket_id) ON DELETE CASCADE,
    message_text TEXT         NOT NULL,
    author       TEXT         NOT NULL,
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_messages_ticket_id ON ticket_messages(ticket_id);
CREATE INDEX idx_tickets_status     ON tickets(status);
CREATE INDEX idx_tickets_priority   ON tickets(priority);
```

Seed Data Verification

Sample data was inserted covering all three statuses with multi-message threads, then verified with a join + aggregate query:

```
SELECT t.ticket_id, t.title, t.status, t.priority,
       COUNT(m.message_id) AS message_count
FROM tickets t
LEFT JOIN ticket_messages m ON t.ticket_id = m.ticket_id
GROUP BY t.ticket_id, t.title, t.status, t.priority
ORDER BY t.ticket_id;
```

ID	Title	Status	Priority	Msgs
1	Cluster auto-terminates during pipeline run	in_progress	high	4
2	Unable to access Unity Catalog schema	in_progress	high	3
3	Delta table OPTIMIZE job taking longer	resolved	medium	3
4	Lakebase connection string not recognized	open	medium	2
5	Spark job failing with OOM error	in_progress	high	1

Result: 5 tickets across 3 distinct statuses, 13 total messages — exceeds the assignment's minimum of 3 tickets / 2 statuses / 2 messages each.

PHASE 2

Database Access Layer (db.py)

A single module isolating all SQL. The application code never writes raw queries — every read/write goes through a typed helper function.

Connection Strategy

Inside Databricks Apps, the platform injects `DATABRICKS_CLIENT_ID` and `DATABRICKS_CLIENT_SECRET` for the app's service principal. `db.py` exchanges these for a short-lived OAuth JWT via the `client_credentials` grant, then opens a psycopg2 connection to Lakebase using that token as the password.

```
def get_oauth_token():
    client_id     = os.environ["DATABRICKS_CLIENT_ID"]
    client_secret = os.environ["DATABRICKS_CLIENT_SECRET"]
    host          = os.environ.get("DATABRICKS_HOST", ...)

    r = requests.post(f"{host}/oidc/v1/token", data={
        "grant_type":    "client_credentials",
        "client_id":     client_id,
        "client_secret": client_secret,
        "scope":         "all-apis",
    })
    return r.json()["access_token"]

def get_conn():
    token     = get_oauth_token()
    client_id = os.environ["DATABRICKS_CLIENT_ID"]
    conn_str  = (f"postgresql://{client_id}:{token}"
                 f"@{LAKEBASE_HOST}/{LAKEBASE_DB}?sslmode=require")
    return psycopg2.connect(conn_str, cursor_factory=RealDictCursor)
```

Functions Exposed

Function	Purpose
`get_all_tickets(status_filter)`	List tickets, optional status filter
`get_ticket_by_id(id)`	Single ticket lookup
`get_messages(ticket_id)`	Full message thread, chronological
`get_stats()`	Counts by status for the dashboard
`create_ticket(title, created_by, priority, category)`	Insert + return new ticket_id
`add_message(ticket_id, text, author)`	Insert a threaded message
`update_status(ticket_id, new_status)`	Update ticket status

`delete_ticket(ticket_id)`	Delete with FK cascade to messages

PHASE 3

Application UI (app.py, Gradio)

A three-tab Gradio interface styled to resemble a corporate helpdesk portal — top navigation bar, sticky sidebar dashboard, and content tabs.

UI Structure

- **All Tickets** — filterable table of every ticket, status/priority/category/message-count columns
- **View & Update** — load a ticket by ID; see a formatted detail card, full message thread, status-update control, and a guarded delete action
- **New Ticket** — validated creation form (title, priority, category, requester)
- A sticky left sidebar shows live counts (Total / Open / In Progress / Resolved) that refresh after every write operation

Design System

Light, neutral corporate palette (slate/indigo) modeled on common helpdesk products (Zendesk / Freshdesk / Jira Service Desk): white cards on a soft slate background, colored status chips, Inter typeface, and consistent 42–44px control heights.

Bonus: Delete Confirmation Guard

```
def on_delete_ticket(tid, confirmed):
    if not tid:       return "No ticket loaded", ...
    if not confirmed: return "Tick the confirmation box first", ...
    delete_ticket(int(tid))
    return f"Ticket #{tid} deleted", ...
```

PHASE 4

Deployment on Databricks Apps

Connected the GitHub repository to a Databricks App, wired the Lakebase secret as a scoped resource, and verified end-to-end connectivity.

Deployment Configuration

App name	support-ticketing
Source	GitHub — demonjd2026-afk/lakebase-support-ticketing, branch main, path app/
Entrypoint	python app.py (set via app.yml)
Secret resource	Databricks secret scope → LAKEBASE_CONN env var
App URL	https://support-ticketing-7474654640109575.aws.databricksapps.com

app.yml

```
command: ["python", "app.py"]

env:
  - name: LAKEBASE_CONN
    valueFrom: LAKEBASE_CONN
```

Access Control

The App's auto-generated service principal was granted the `databricks_superuser` role on the Lakebase branch via *Roles & Databases*, so the app can read and write `tickets` / `ticket_messages` using its own OAuth identity — no personal access token stored anywhere.

Smoke Test Results

- ✓ Existing tickets load correctly from Lakebase on page open
- ✓ New ticket created via the form persists and appears in the list
- ✓ Message added to a ticket persists and appears in the thread
- ✓ Ticket status updated and reflected immediately, survives a page refresh
- ✓ Status filter narrows the ticket list correctly
- ✓ Delete requires the confirmation checkbox before removing a ticket

6 · Screenshots — Live Application Walkthrough

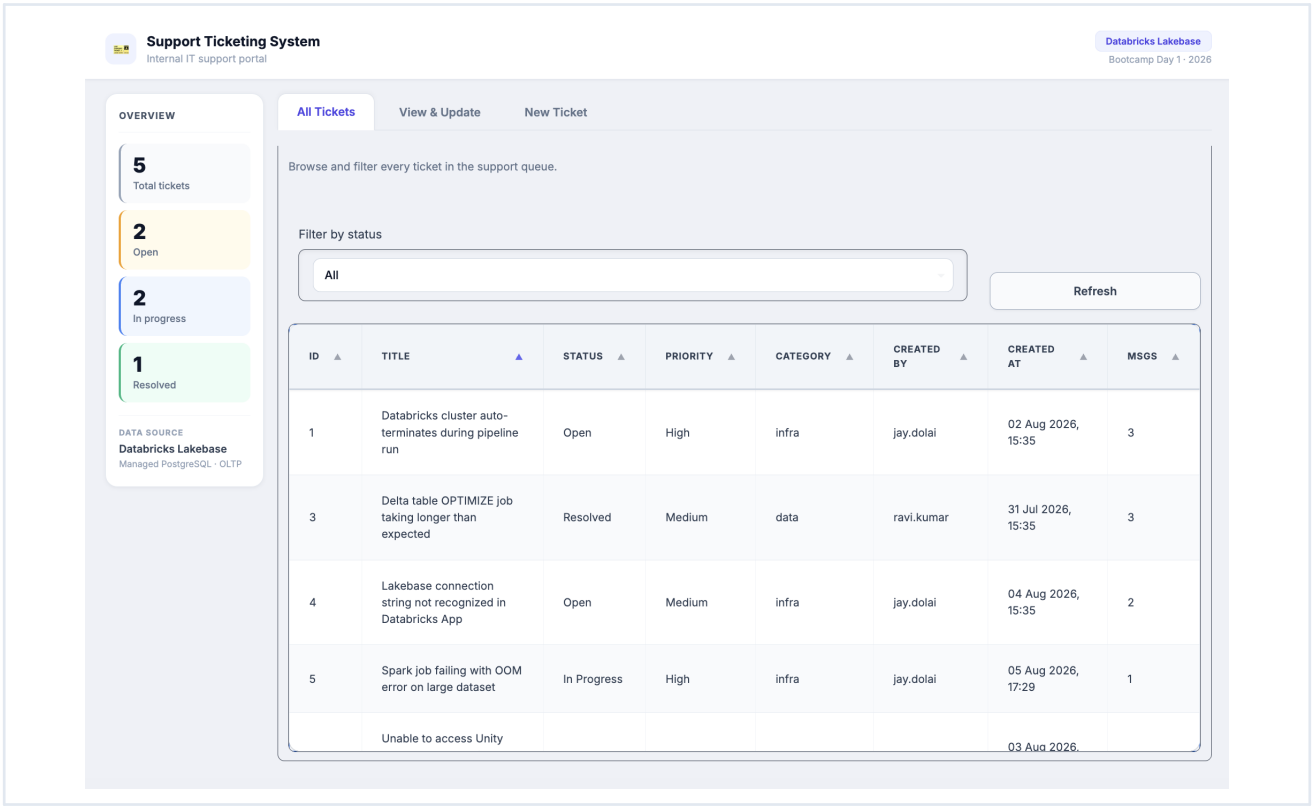


Fig. 1 — All Tickets tab: live dashboard sidebar (Total / Open / In Progress / Resolved) and the full ticket table.

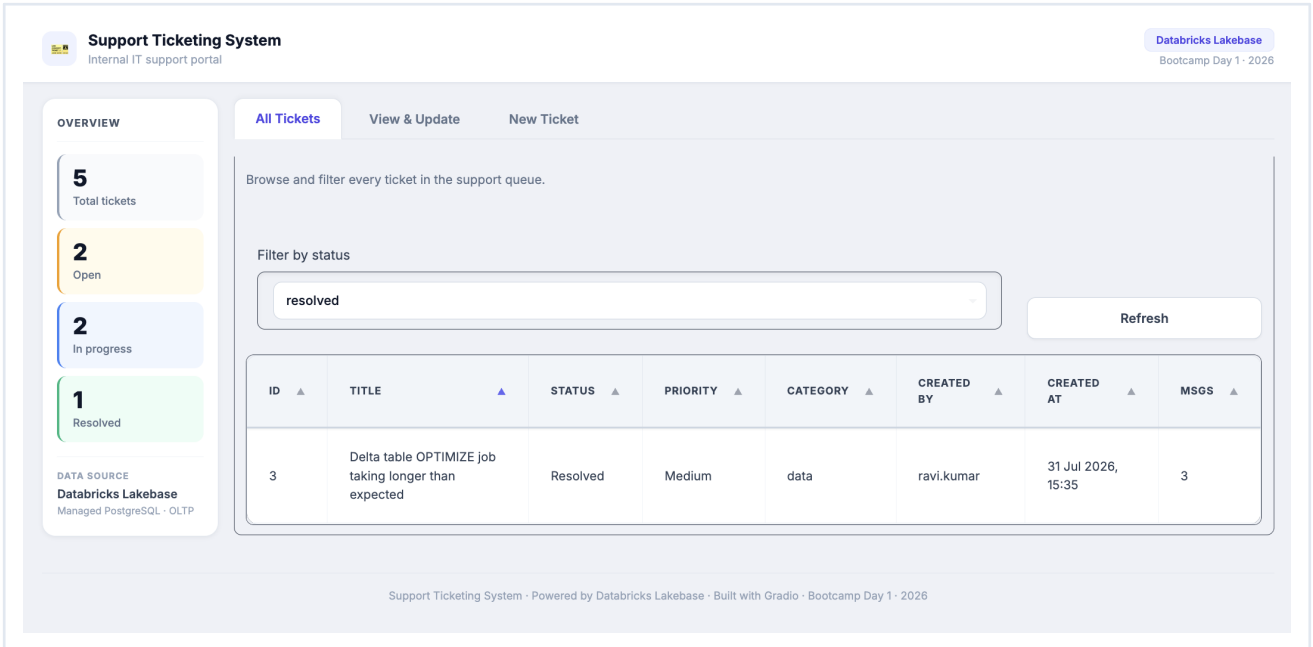


Fig. 2 — Filter by status: narrowing the queue to resolved tickets only.

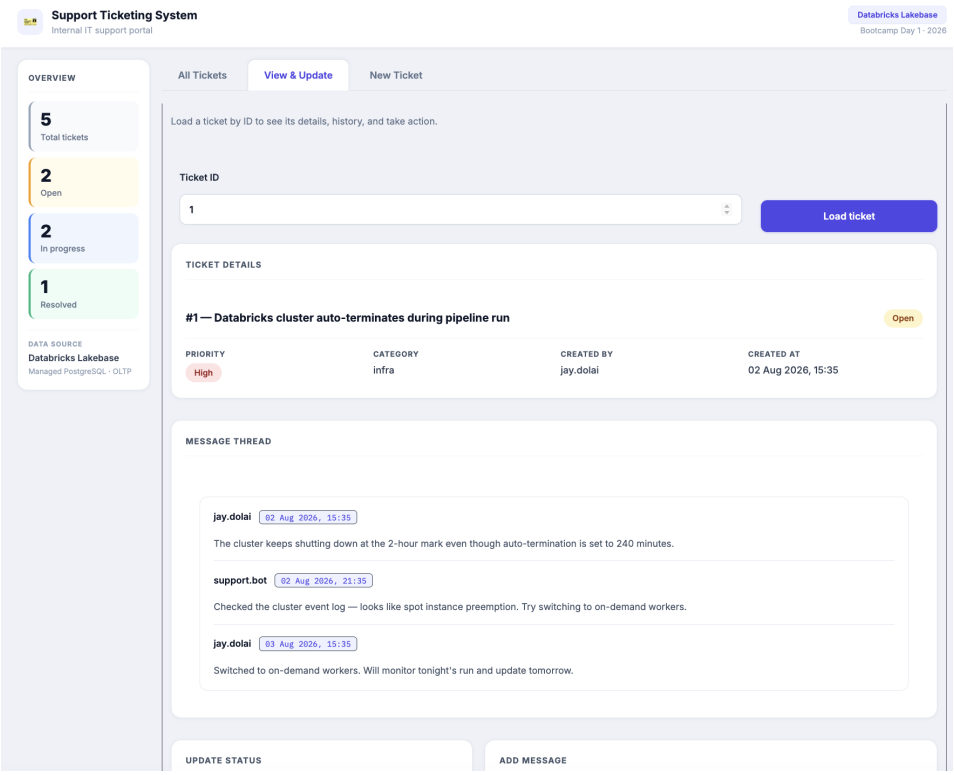


Fig. 3 — View & Update tab: ticket detail card with status/priority chips and full message thread.

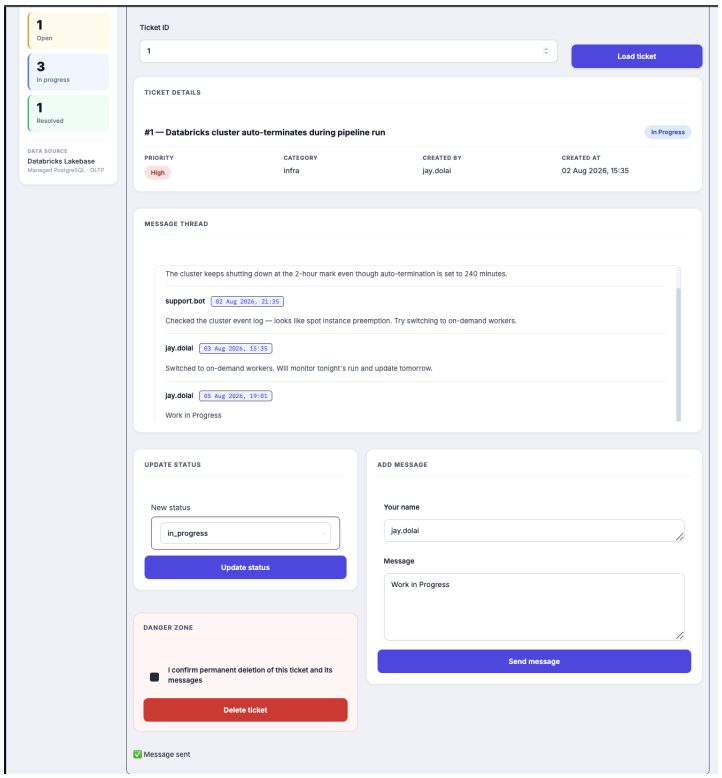


Fig. 4 — Add Message: a new reply appended to the thread, confirmed with inline feedback.

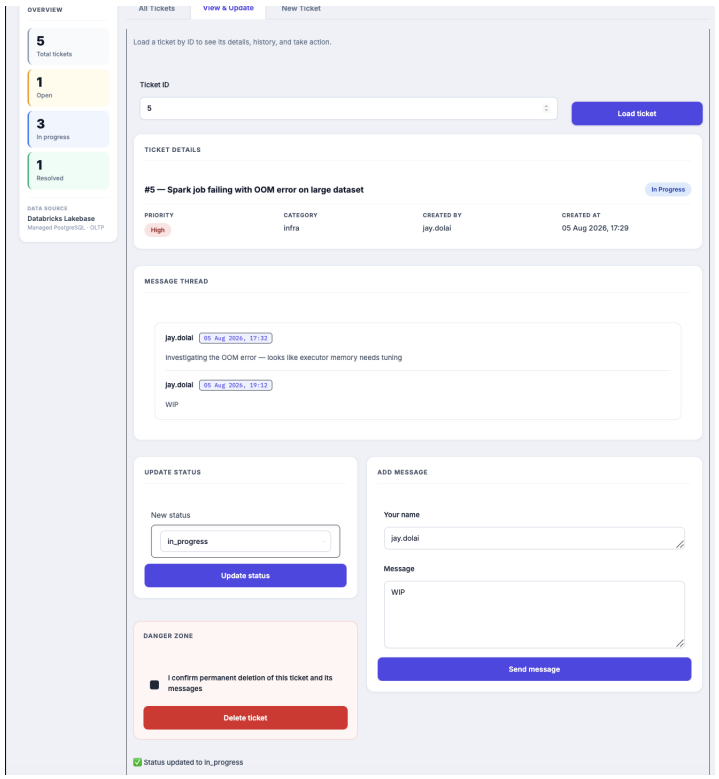


Fig. 5 — Update Status: ticket status changed to in_progress, reflected immediately in the detail card.

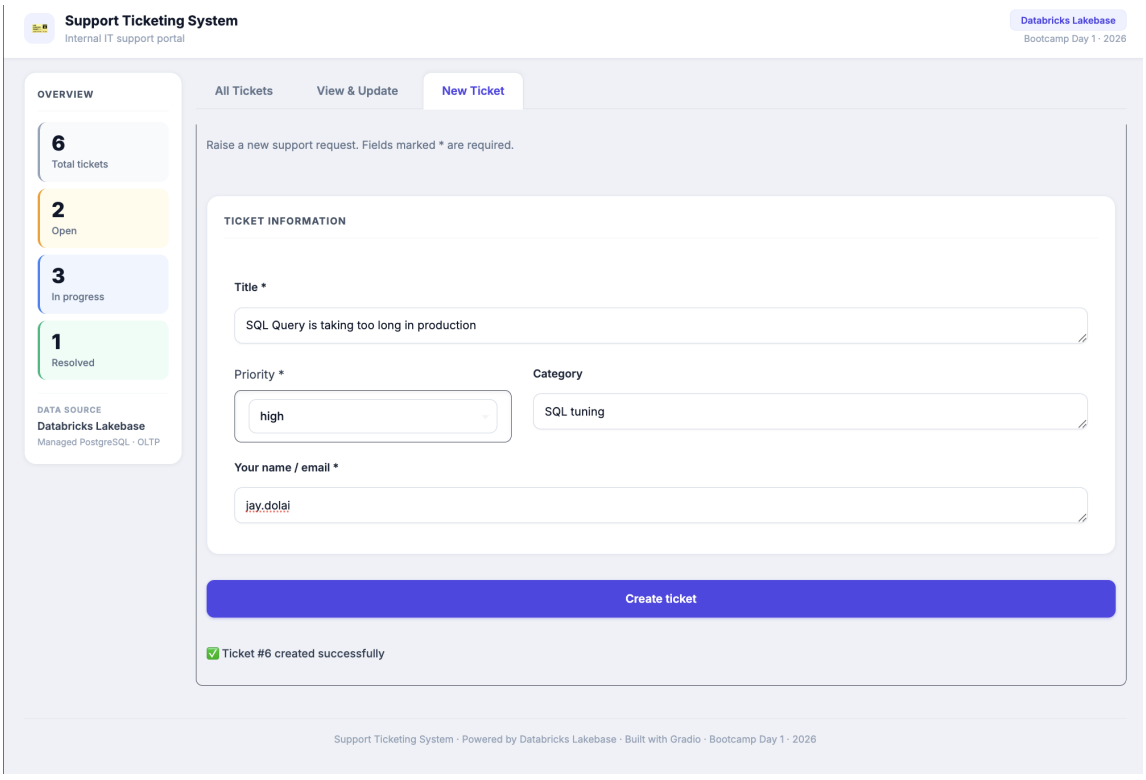


Fig. 6 — New Ticket tab: validated creation form; ticket #6 created and the sidebar total increments to 6.

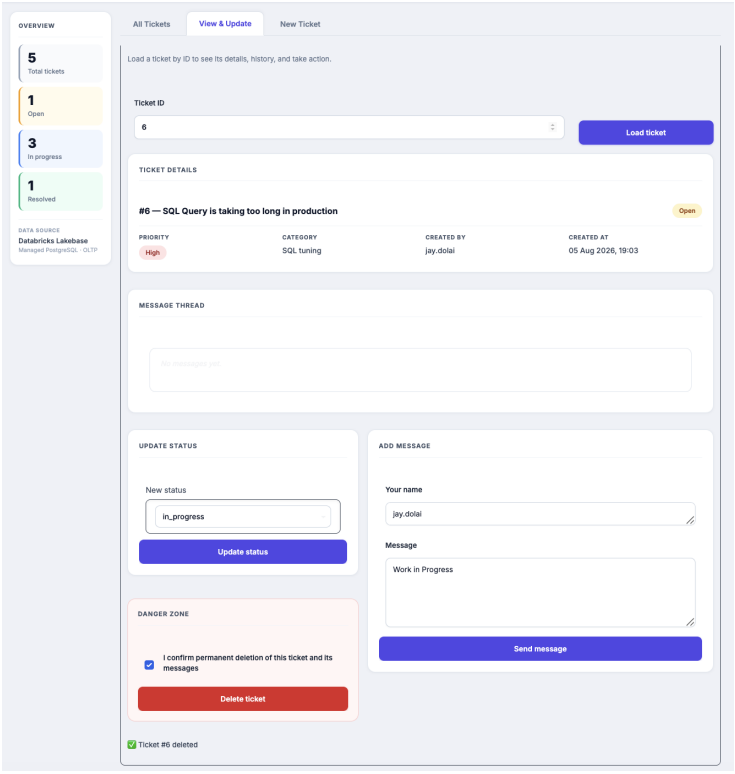


Fig. 7 — Delete with confirmation: the checkbox must be ticked before Delete Ticket is accepted; ticket #6 removed.

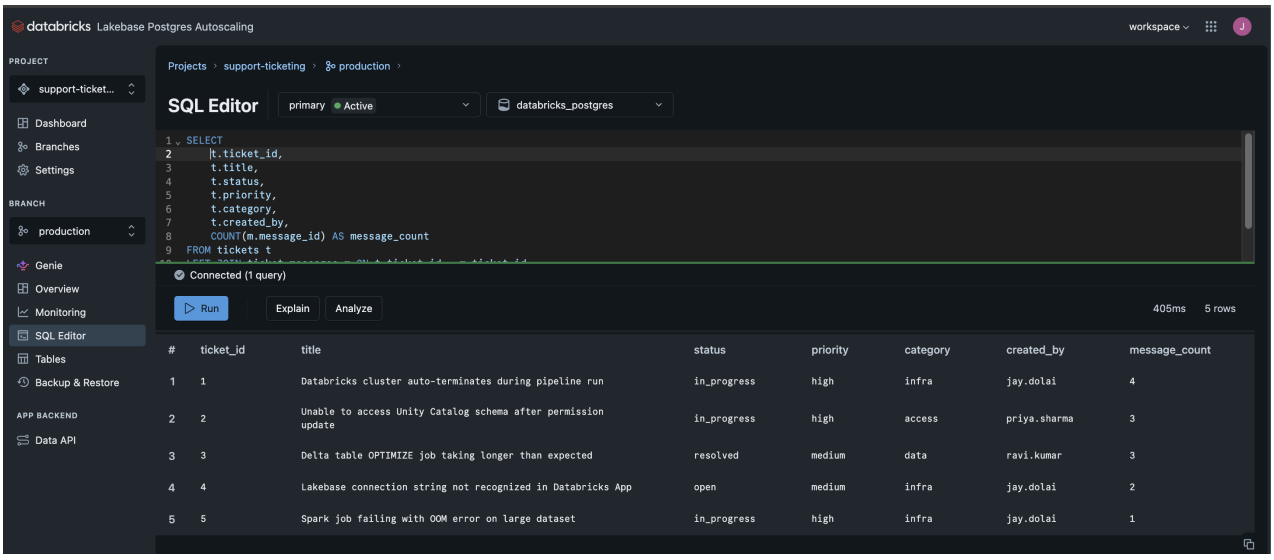


Fig. 8 — Lakebase SQL Editor: verification query showing tickets joined with message counts, run directly against the production branch.

7 · Bonus Features Implemented

- ✓ **Priority & category** — both fields present in schema, forms, and table columns
- ✓ **Filter by status** — dropdown on the All Tickets tab (All / open / in_progress / resolved)
- ✓ **Input validation** — required-field checks with inline error messages on create/update/message actions
- ✓ **Ticket statistics** — live sidebar dashboard (Total / Open / In Progress / Resolved), refreshed after every mutation
- ✓ **Delete with confirmation** — explicit checkbox required before the delete action is accepted
- ✓ **Improved visual design** — corporate helpdesk-style UI with custom CSS, status/priority chips, and a sticky dashboard sidebar

8 · Reflection

What was the most difficult part?

Getting the Databricks App to authenticate against Lakebase. Lakebase's OAuth connections require a JWT access token — not a personal access token — so the working path was to have the app's own service principal exchange its injected `DATABRICKS_CLIENT_ID`/`DATABRICKS_CLIENT_SECRET` for a JWT via the OIDC `client_credentials` grant, and to grant that principal a Lakebase role on the branch. Several intermediate approaches (raw PAT, hardcoded connection strings, mismatched hostnames) failed with distinct errors before the correct service-principal-OAuth flow was identified.

How is Lakebase different from a traditional analytics table?

Lakebase is managed PostgreSQL exposed over the standard wire protocol, giving row-level ACID transactions and millisecond read/write latency — exactly what an interactive app doing single-row inserts and updates needs. A Delta table is optimized for large, append-oriented batch writes and file-level commits; using it for a live app's create/update/delete operations would be both slow and awkward compared to Lakebase's transactional model.

What feature would you add next?

An AI agent layer on top of the same Lakebase tables — using Databricks Model Serving (e.g. Llama or Claude) to auto-triage new tickets (suggest priority/category), draft reply messages, and summarize a long thread on load, mirroring the agentic pattern already used in the AI Stock Market Research Assistant capstone.

9 · Submission Checklist

- ✓ Databricks App URL — https://support-ticketing-7474654640109575.aws.databricksapps.com
- ✓ Source code — github.com/demonjd2026-afk/lakebase-support-ticketing
- ✓ Screenshot of the deployed application — included above
- ✓ Screenshot of the Lakebase tables and sample records — included above
- ✓ Reflection (3–5 sentences) — included above